

- VERSIONING
- > Changelog
- Upgrade your API version
- > Upgrade your SDK version

- ESSENTIALS
- > SDKs
- > API
- Overview
- > API v2
- Include-dependent response values v2
- Represent customers using Account objects
- Rate limits
- AUTHENTICATION
- > API keys
- Specify request context
- Domains and IP addresses
- MAKE REQUESTS
- > Expand responses
- Pagination
- Search objects
- Localise content
- TESTING AND DATA
- > Metadata
- Test your application
- ERROR HANDLING
- > Handle errors
- Error codes
- > Testing
- > Stripe CLI

- TOOLS
- > Stripe Dashboard
- > Workbench
- > Developers Dashboard
- Stripe for Visual Studio Code
- > Terraform
- Stripe Discord server

- FEATURES
- > Workflows
- > Batch Jobs
- > Event Destinations

API v2 overview

Understand the behaviour of APIs in the v2 namespace.

View this page as plain text

- Ask about this page
- Copy for LLM
- View as Markdown
- Install tools

The Stripe API provides two namespaces that contain different sets of endpoints:

- API v1: The `/v1` namespace includes most of the existing Stripe API today.
- API v2: The `/v2` namespace includes endpoints that use `/v2` design patterns.

Key differences between the v1 and v2 namespace

	API V1	API V2
Send data to the API	Requests use form encoding (<code>application/x-www-form-urlencoded</code>) and responses use JSON encoding (<code>application/json</code>).	Request and responses use JSON encoding (<code>application/json</code>).
Test your integration	Validate APIs in the <code>/v1</code> namespace using Sandboxes, an isolated environment.	Validate APIs in the <code>/v2</code> namespace using Sandboxes, an isolated environment. Read more: Sandboxes
Send idempotent requests	When providing the <code>Idempotency-Key</code> header with a unique identifier, if the API already processed the request, it returns the previously stored request.	When providing the <code>Idempotency-Key</code> header with a unique identifier, the API retries any failed requests without producing side effects (any extraneous change or observable behaviour that occurs as a result of an API call). Read more: Idempotency
Receive events from Stripe	Most events emitted from APIs in the <code>/v1</code> namespace include a snapshot of an API object in their payload. Some APIs in the <code>/v1</code> namespace generate thin events, which include a minimal, unversioned push payload.	Events emitted from APIs in the <code>/v2</code> namespace are <code>thin events</code> . Read more: Event destinations
Paginating through a list	Specify an object's ID as the starting element for list API requests. Use the <code>starting_after</code> , <code>ending_before</code> and <code>has_more</code> properties from the API response to paginate through a list.	Specify the <code>page</code> token for list API requests. Use the <code>previous_page_url</code> and <code>next_page_url</code> properties in the API response to paginate through a list. Read more: List pagination
Consistency guarantees for lists	Top-level lists are immediately consistent (with higher latency to render). Some sublists are eventually consistent.	Lists are eventually consistent by default and lower-latency.
Fetch additional data with expansion	Use the <code>expand</code> parameter to replace IDs for related API objects with fully-expanded child objects. Read more: Expanding responses	The <code>expand</code> parameter isn't supported. Some APIs in this namespace might provide additional fields in their responses by using the <code>include</code> parameter.

	API V1	API V2
Manage metadata	Remove a key-value pair by setting the value to an empty string.	Remove a key-value pair by setting the value to <code>null</code> .

SDKs that support API v2

All server-side SDKs support APIs in the `/v2` namespace.

Using API v2 with the Stripe CLI

Use `stripe trigger` and `stripe listen` to test your integration’s event handling.

To access APIs in the `/v2` namespace using the Stripe CLI, use the command `stripe v2`. For example, to list all `v2 Accounts`, you can use `stripe v2 core accounts list`.

SDK, CLI and API versioning

SDKs and the Stripe CLI automatically include an API version for all requests. After you update your SDK or CLI version, Stripe simultaneously updates the API version of your requests and responses.

Include Stripe-Version without SDK or CLI

All API requests to the API `/v2` namespace must include the `Stripe-Version` header to specify the underlying API version.

For example, a curl request using API version `2024-09-30.acacia` looks like:

📄 Command Line ⓘ 📋 ✨

```
$ curl -G https://api.stripe.com/v2/core/event_destinations \
> -H "Authorization: Bearer {{YOUR_API_KEY}}" \
> -H "Stripe-Version: 2024-09-30.acacia" \
```

Using APIs from the v1 and v2 namespaces in the same integration

You can use any combination of APIs in the `/v1` or `/v2` namespace in the same integration.

Java ⓘ 📋 ✨

```
1 import com.stripe.StripeClient;
2
3 StripeClient stripe = new StripeClient("{{YOUR_API_KEY}}");
4
5 // Call a v2 API
6 EventDestination eventDestination = stripe.v2().core().eventDestinations().retrieve("ed_1
7
8 // Call a v1 API
9 Customer customer = stripe.customers().retrieve("cus_123");
```

If you’re not using an official SDK or the CLI, always include the namespace in the URL path for your API calls. For example:

📄 Command Line ⓘ 📋 ✨

```
# Call a v2 API
curl https://api.stripe.com/v2/core/event_destinations

# Call a v1 API
curl https://api.stripe.com/v1/charges -d amount=2000 -d currency=usd
```

List pagination

APIs within the `/v2` namespace (for example, `GET /v2/core/event_destinations`) contain a different pagination interface compared to those in the `/v1` namespace.

The `previous_page_url` property returns a URL to fetch the previous page of the list. If there are no previous pages, the value is `null`.

The `next_page_url` property returns a URL to fetch the next page of the list. If there are no more pages, the value is `null`.

You can use these URLs to make requests without using our SDKs. Conversely, when you use our SDKs, you don't need to use these URLs because the SDKs handle auto-pagination automatically.

You can't change list filters after the first request.

Java ⓘ 📄 ✨

```
1 StripeClient stripe = new StripeClient("{}YOUR_API_KEY{}");
2
3 EventDestinationListParams params = EventDestinationListParams.builder().build();
4
5 for (EventDestination eventDestination : stripe.v2().core().eventDestinations().list(params)) {
6     // process event destination object
7 }
```

Using query parameters in API requests

Use filters on list endpoints to constrain results. Use `include` parameters on supported GET endpoints to return additional fields in the response. Filters and include parameters let you pass an array of values. When you make direct API requests (not using server-side SDKs or the CLI), you must always specify the index of the array value using bracket notation, even if you only pass a single value.

For example, to list all `Accounts` where the `applied_configuration` is `merchant`, pass the following:

🔗 Command Line

ⓘ 📄 ✨

```
$ curl -G https://api.stripe.com/v2/core/accounts?applied_configurations[0]=merchant
> -H "Authorization: Bearer {}YOUR_API_KEY{}" \
> -H "Stripe-Version: 2025-12-15.clover" \
```

To list all `Accounts` where the configuration is `merchant` or `customer`, use the following syntax:

🔗 Command Line

ⓘ 📄 ✨

```
$ curl -G https://api.stripe.com/v2/core/accounts?applied_configurations[0]=merchant&applied_configurations[1]=customer
> -H "Authorization: Bearer {}YOUR_API_KEY{}" \
> -H "Stripe-Version: 2025-12-15.clover" \
```

You can use the same pattern for include parameters. For example, to include multiple fields in a response, use the following syntax:

🔗 Command Line

ⓘ 📄 ✨

```
$ curl -G https://api.stripe.com/v2/core/accounts/:id?include[0]=requirements&include[1]=default_source
> -H "Authorization: Bearer {}YOUR_API_KEY{}" \
> -H "Stripe-Version: 2025-12-15.clover" \
```

In some cases, an array is nested within a query parameter. For example, to retrieve a list of payout methods, you can provide an array of values to filter on the `usage_status[payments]` nested field:

🔗 Command Line

ⓘ 📄 ✨

```
$ curl -G https://api.stripe.com/v2/money_management/payout_methods?usage_status[payments] [
>   -H "Authorization: Bearer {{YOUR_API_KEY}}" \
>   -H "Stripe-Version: 2025-12-15.preview" \
```

Idempotency

APIs in the `/v2` namespace provide improved support for idempotency behaviour, preventing unintended side effects when requests are performed multiple times using the same idempotency key. When the API receives two requests with the same idempotency key:

If the first request succeeded, the API skips making new changes and returns an updated response.

If the first request failed (or partially failed), the API re-executes the failed requests and returns the new response.

In the rare event that it's no longer possible for an idempotent replay to succeed, the API returns an error explaining why.

A request is considered an idempotent replay of another request if the following are all true:

They use the same idempotency key for the same API

They occur in the scope of the same account or sandbox

They occur within 30 days of each other

To specify an idempotency key, use the `Idempotency-Key` header and provide a unique value to represent the operation (we recommend a UUID). If no key is provided, Stripe automatically generates a UUID for you.

All `POST` and `DELETE` API v2 requests accept idempotency keys and behave idempotently. `GET` requests are idempotent by definition, so sending an idempotency key has no effect.

Idempotency differences between API v1 and API v2

API v1 and API v2 idempotency have a few key differences:

API v1 only supports idempotent replay for `POST` requests. API v2 supports all `POST` and `DELETE` requests.

A request is considered an idempotent replay of another request for:

API v1 if they use the same idempotency key and occur within 24 hours of each other.

API v2 if they use the same idempotency key, are made to the same API, occur within the scope of the same account or sandbox and occur within 30 days of each other.

When you provide the same idempotency key for two requests:

API v1 always returns the previously-saved response of the first API request, even if it was an error.

API v2 attempts to retry any failed requests without producing side effects (any extraneous change or observable behaviour that occurs as a result of an API call) and provide an updated response.

Making idempotent requests

Using the SDK, provide an idempotency key with the `idempotencyKey` property in API requests.

For example, to make an API request with a specific idempotency key:

Java



```
1 StripeClient stripe = new StripeClient("{}YOUR_API_KEY{}");
2
3 String idempotencyKey = "unique-idempotency-key";
4 Example result = stripe.v2().examples().create(
5     ExampleCreateParams.builder()
6         .setName("My example")
7         .build(),
8
9     RequestOptions.builder()
10        .setIdempotencyKey(idempotencyKey)
11        .build());
```



If you're not using a SDK or the CLI, requests can include the `Idempotency-Key` header:

Command Line



```
$ curl https://api.stripe.com/v2/examples \
> -H "Authorization: Bearer {}YOUR_API_KEY{}" \
> -H "Stripe-Version: {}STRIPE_API_VERSION{}" \
> -H "Idempotency-Key: unique-idempotency-key" \
> -d <JSON request body>
```

Limitations

Not all `/v2` APIs support `test mode sandboxes`. You can always test `/v2` endpoints in a sandbox.

Currently, Stripe only generates thin events using `/v2` endpoints and resources.

You can only see request logs generated by API v2 in [Workbench](#), not in the Developers Dashboard.

